



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ
КАФЕДРА

Информатика и системы управления
Теоретическая информатика и компьютерные технологии

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Сравнительный анализ

нормализованной и документно-ориентированной

моделей хранения данных

системы видеонаблюдения

Студент

ИУ9-62Б

(группа)

(подпись, дата)

Федуков А.А.

(И.О. Фамилия)

Руководитель курсовой
работы

(подпись, дата)

Домрачева А.Б.

(И.О. Фамилия)

Оценка

2026 г.

Содержание

Введение	4
1 Анализ предметной области	6
1.1 Общая характеристика системы видеонаблюдения	6
1.2 Сценарий работы системы	7
2 Проектирование модели данных	8
2.1 Описание сущностей	8
2.1.1 Area	8
2.1.2 Zone	9
2.1.3 Camera	10
2.1.4 Event	10
2.1.5 CameraTelemetry	11
2.1.6 Итоговый набор сущностей	12
2.2 Описание связей между сущностями	13
2.3 ER-диаграмма	14
3 Проектирование моделей хранения	15
3.1 Документно-ориентированная модель PostgreSQL JSONB	15
3.1.1 Ключи и связи	15
3.1.2 Использование JSONB	16
3.1.3 Особенности модели	16
3.2 Нормализованная реляционная модель PostgreSQL	18
3.2.1 Нормализация справочных значений	18
3.2.2 Декомпозиция составных атрибутов	18
3.2.3 Нормальная форма	19
3.3 Вложенная документная модель MongoDB	22
3.4 Нормализованная документная модель MongoDB	23

4	Реализация баз данных и веб-приложения	24
4.1	Проектирование программной системы	24
4.1.1	Общая архитектура	24
4.2	Создание баз данных	25
4.2.1	Инициализация схем	25
4.2.2	Индексы	25
4.2.3	Подготовка данных	26
4.3	Реализация симулятора	26
4.3.1	Генератор нагрузки	26
4.3.2	Метрики loadgen	27
4.3.3	Операции нагрузки	27
4.3.4	Характер запросов	29
4.4	Реализация веб-приложения	31
4.4.1	Пользовательский интерфейс	31
4.4.2	Отображение метрик	31
5	Экспериментальное сравнение моделей	31
5.1	Сравнение времени записи	31
5.2	Сравнение времени чтения	34
5.3	Сравнение моделей при разной нагрузке	35
5.4	Анализ полученных результатов	37
	Заключение	39
	Источники	40

Введение

На сегодняшний день практически любое приложение использует базу данных для хранения и обработки информации. Неудивительно, что существует множество подходов к организации системы хранения данных. Различные решения могут отличаться по модели представления информации, способам обеспечения целостности, масштабируемости и производительности. Выбор конкретного варианта зачастую является неочевидной задачей, поскольку преимущества одной модели хранения данных в одних сценариях могут сопровождаться ухудшением характеристик в других. Например, высокая эффективность при записи данных не всегда сочетается с удобством и скоростью их извлечения, а гибкость структуры хранения может достигаться ценой увеличения сложности поддержки целостности всей системы данных.

Для каждого приложения необходимо подобрать модель хранения данных, которая будет соответствовать его требованиям и обеспечивать оптимальный баланс между производительностью, удобством использования и поддержкой. Тем не менее часто можно руководствоваться общими рекомендациями. Так, например, реляционные базы данных с нормализованной структурой хранения данных традиционно считаются более подходящими для приложений, требующих строгой целостности данных и сложных связей между сущностями, а документно-ориентированные базы данных, допускающие более гибкую структуру хранения данных, могут лучше подходить для приложений, где структура данных может часто меняться.

Однако, чтобы гарантировать преимущества и недостатки различных моделей хранения данных в конкретной предметной области, надежнее всего сравнить итоговую производительность в реализациях на разных моделях или даже СУБД.

Примером такой ”неочевидной” предметной области может служить система видеонаблюдения, включающая большое количество камер. Они формируют большой поток информации в реальном времени. Причем данные, которые необходимо хранить, могут быть разнородными и также часто меняться, к примеру метаданные о событиях и объектах, обнаруженных на видео, или информацию о состоянии камер и их конфигурации. При эксплуатации подобных систем не ясно, какая модель хранения данных будет более эффективной в таких условиях. Для выбора конкретного варианта следует провести исследование особенностей хранения данных системы видеонаблюдения, снять основные показатели производительности и сравнить их для разных моделей хранения данных, чтобы сделать выбор в пользу одной из них.

Целью данной работы является сравнительный анализ нормализованной и документно-ориентированной моделей хранения данных системы видеонаблюдения на примере СУБД PostgreSQL и MongoDB. Соответственно, необходимо четко определить критерии сравнения и сценарии использования. После чего провести нагрузочное тестирование обеих моделей хранения данных, оценить итоговую производительность и сделать выводы о том, какая модель хранения данных более эффективна для системы видеонаблюдения.

1. Анализ предметной области

Для сравнения моделей хранения необходимо зафиксировать предметную область. Модель должна быть близкой к реальным системам видеонаблюдения, но без лишних деталей, которые бы влияли на хранение и анализ метаданных.

1.1. Общая характеристика системы видеонаблюдения

Современные системы видеонаблюдения используются не только для визуального контроля территории. Часто они служат источником различных данных, которые могут быть использованы для анализа событий. В общем случае такая система включает камеры, зоны наблюдения, серверы обработки или хранения, программное обеспечение для управления потоками данных и анализа видеозаписей. Согласно руководствам по проектированию CCTV-систем, при построении системы важно учитывать качество изображения, размещение камер, а также механизмы передачи, обработки и хранения данных [1].

Видеопоток остается основным типом данных, которые собирает система видеонаблюдения. Однако помимо видеоданных системы генерируют множество метаданных и событий, которые могут быть не менее важными для анализа. Например, умные камеры могут распознавать объекты, определять их тип (человек, автомобиль, животное), отслеживать их перемещение, фиксировать время и место событий. Эти данные используют для поиска закономерностей, обнаружения аномалий и других задач, которые выходят за рамки простого просмотра видеозаписей.

В рамках данной работы система видеонаблюдения будет рассматриваться не как полноценный программно-аппаратный комплекс охраны, а как ее часть. Поэтому исследование фокусируется на метаданных и ис-

ключает хранение самого видеопотока.

Система строится вокруг хранения и анализа событий, полученных от большого количества камер. Камеры фиксируют происходящее в зонах наблюдения, а встроенная в камеру система аналитики формирует события: обнаружение движения, появление человека или автомобиля, пересечение виртуальной линии, нахождение объекта в запрещенной зоне, потерю сигнала с камеры и другие ситуации. Для каждого события сохраняются метаданные, необходимые для последующего поиска, фильтрации и анализа.

Эти метаданные загружаются в СУБД, которая обеспечивает их хранение, индексацию и доступ для аналитических задач.

1.2. Сценарий работы системы

Рассмотрим сценарий: имеется охраняемая территория, разделенная на несколько зон наблюдения. В каждой зоне установлены камеры. Каждая камера передает данные в локальное хранилище, расположенное на охраняемой территории. Данные внутри этого хранилища обрабатываются и анализируются, итоговая информация сохраняется на удаленном сервере в базе данных. Система не сохраняет на удаленном сервере видеозаписи, но фиксирует сведения о состоянии камер и событиях, полученных в результате внутренней аналитики.

Основной поток данных образуют события видеонаблюдения. Событие представляет собой факт, зафиксированный камерой или аналитической системой (внутри камеры или на локальной системе), который может быть так или иначе сохранен в локальном хранилище охраняемой территории. Оттуда, в любом случае, итоговое событие передается на общий сервер, где сохраняется в базе данных.

Например, система может обнаружить движение, потерю сигнала,

появление объекта или пересечение заданной линии в кадре. При этом для каждого события сохраняются дата и время возникновения, зона, одна или несколько камер, зафиксировавших событие, тип события, уровень значимости, уверенность аналитического алгоритма и дополнительные параметры.

Нас интересуют именно эти метаданные, приходящие потоком на удаленный сервер. Важно зафиксировать их структуру, объем, частоту и другие характеристики, чтобы использовать равные условия для сравнения моделей данных.

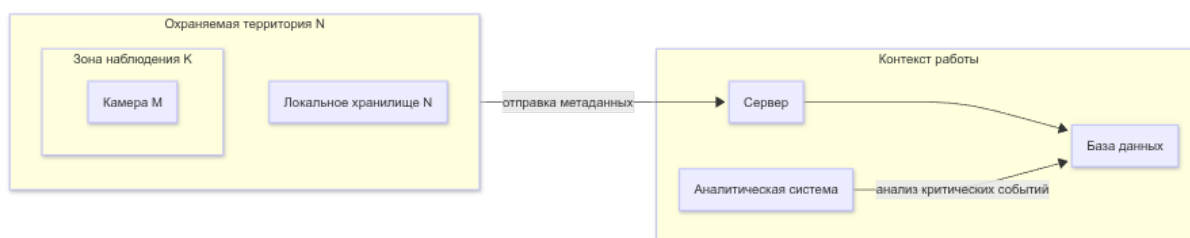


Рисунок 1.1 – Сценарий работы системы

2. Проектирование модели данных

Итак, наша модель фокусируется на сервере, который принимает обработанные данные от множества камер.

2.1. Описание сущностей

Модель включает пять сущностей: **Area**, **Zone**, **Camera**, **Event** и **CameraTelemetry**.

2.1.1. Area

Area — охраняемая территория, на которой развернута система видеонаблюдения.

Основные атрибуты территории:

- area_code — предметный идентификатор территории;
- name — название территории;
- area_type — тип территории;
- address — адрес или описание местоположения;
- description — дополнительное описание территории.

Одна территория может содержать несколько зон наблюдения.

2.1.2. Zone

Zone — часть охраняемой территории, внутри которой установлены камеры и фиксируются события.

Основные атрибуты зоны:

- zone_code — идентификатор зоны внутри территории;
- area_code — идентификатор территории, к которой относится зона;
- name — название зоны;
- zone_type — тип зоны;
- importance_level — уровень важности зоны;
- description — дополнительное описание зоны.

Идентификатор зоны является составным: зона именуется кодом территории и кодом зоны внутри этой территории. Такой вариант соответствует идентификационно-зависимой сущности: зона не рассматривается отдельно от территории, к которой она относится.

2.1.3. Camera

Camera — камера видеонаблюдения, установленная в зоне.

Основные атрибуты камеры:

- serial_number — предметный идентификатор камеры;
- name — название камеры;
- model — модель камеры;
- ip_address — сетевой адрес камеры;
- status — текущее состояние камеры;
- position — положение и ориентация камеры внутри зоны;
- settings — настройки видеопотока и аналитики.

Камера относится к одной зоне наблюдения. При этом одна зона может содержать несколько камер. Внутренние зоны детекции камеры, используемые алгоритмами аналитики, не являются зонами предметной области и входят в состав атрибута settings.

2.1.4. Event

Event — аналитическое событие, возникшее в зоне наблюдения.

Основные атрибуты события:

- event_number — номер события внутри зоны;
- zone_code — идентификатор зоны, в которой произошло событие;
- area_code — идентификатор территории события;
- occurred_at — время возникновения события;

- `event_type` — тип события;
- `severity` — уровень значимости события;
- `confidence` — уверенность аналитического алгоритма;
- `payload` — составные данные события.

Идентификатор события является составным: номер события уникален внутри зоны, а зона определяется кодом зоны и кодом территории. Событие может быть зафиксировано одной или несколькими камерами. Например, один факт пересечения линии может подтверждаться двумя камерами, установленными в одной зоне.

Атрибут `payload` содержит параметры, зависящие от типа события: параметры движения, сведения о пересеченной линии, информацию о потере сигнала или массив обнаруженных объектов. В ER-модели это составной вариативный атрибут. В разных физических моделях он реализуется по-разному: как `jsonb`, как набор нормализованных таблиц или как вложенный документ.

2.1.5. CameraTelemetry

CameraTelemetry — регулярная запись технического состояния камеры. Эта сущность хранит историю измерений, а не только текущее состояние камеры.

Основные атрибуты телеметрии:

- `serial_number` — идентификатор камеры;
- `recorded_at` — время фиксации телеметрии;
- `status` — состояние камеры на момент фиксации;
- `metrics` — составные технические показатели камеры.

Идентификатор записи телеметрии состоит из идентификатора камеры и времени измерения. Атрибут `metrics` включает технические показатели, например температуру, загрузку процессора, использование памяти, битрейт, потери пакетов, задержку и время непрерывной работы.

2.1.6. Итоговый набор сущностей

Итоговый набор сущностей ER-модели:

- **Area** — охраняемая территория;
- **Zone** — зона наблюдения внутри территории;
- **Camera** — камера видеонаблюдения;
- **Event** — аналитическое событие;
- **CameraTelemetry** — запись технического состояния камеры.

2.2. Описание связей между сущностями

Таблица 2.1. Связи между сущностями предметной области

Сущность 1	Сущность 2	Тип	Связь	Описание
Area	Zone	1:N	M:O	Территория может содержать зоны; зона должна относиться к территории.
Zone	Camera	1:N	M:O	Зона может содержать камеры; камера должна относиться к зоне.
Zone	Event	1:N	M:O	Зона может содержать события; событие должно относиться к зоне.
Camera	Event	M:N	O:M	Камера может фиксировать события; событие должно быть зафиксировано одной или несколькими камерами.
Camera	Camera Telemetry	1:N	M:O	Камера может иметь записи телеметрии; запись телеметрии должна относиться к камере.

2.3. ER-диаграмма

ER-диаграмма (рис. 2.1) строится как концептуальное описание предметной области: она фиксирует сущности, их атрибуты и связи до выбора конкретной СУБД. Такой уровень проектирования разделяет логическое и физическое представления данных [2].

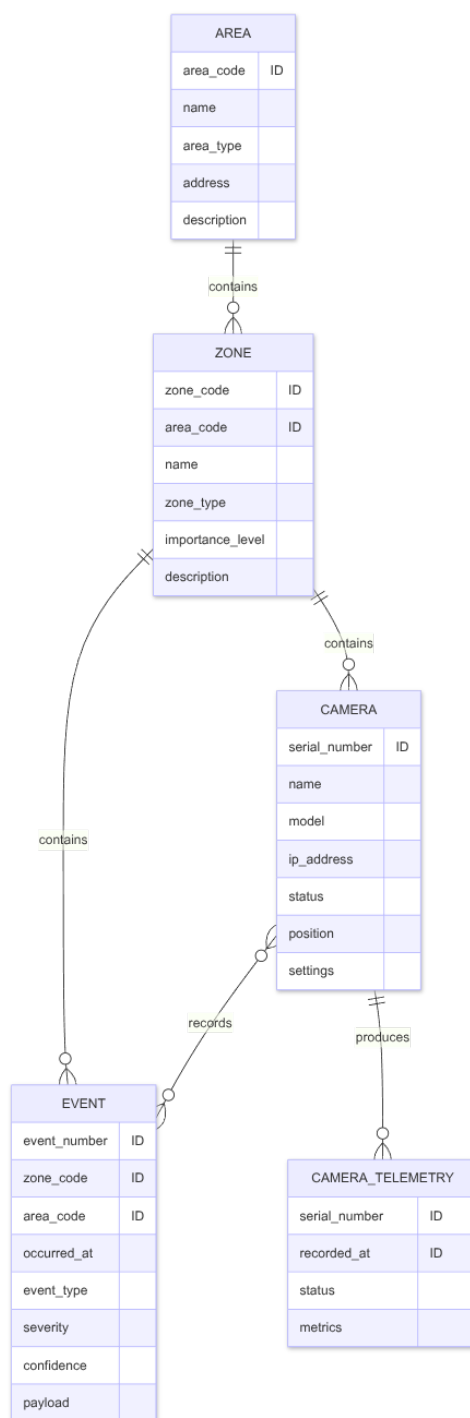


Рисунок 2.1 – ER диаграмма

3. Проектирование моделей хранения

3.1. Документно-ориентированная модель PostgreSQL JSONB

PostgreSQL JSONB-модель строится как частично нормализованная реляционная схема. Основные сущности ER-модели сохраняются в отдельных таблицах: `area`, `zone`, `camera`, `event` и `camera_telemetry`. Связь многие-ко-многим между событиями и камерами реализуется промежуточной таблицей `event_camera`. Для вариативных атрибутов используется тип `jsonb`, поддерживаемый PostgreSQL [3].

3.1.1. Ключи и связи

В таблицах используются суррогатные первичные ключи, удобные для физического хранения и ссылочной целостности. Предметные идентификаторы ER-модели сохраняются как альтернативные ключи:

- `area.area_code` — код территории;
- пара `zone.area_id`, `zone.zone_code` — код зоны внутри территории;
- `camera.serial_number` — серийный номер камеры;
- пара `event.zone_id`, `event.event_number` — номер события внутри зоны;
- пара `camera_telemetry.camera_id`, `camera_telemetry.recorded_at` — запись телеметрии камеры.

Связи один-ко-многим реализуются внешними ключами. Связь **Camera–Event** реализуется таблицей `event_camera`, содержащей идентификатор события и идентификатор камеры.

3.1.2. Использование JSONB

Особенность данной модели заключается в том, что составные и вариативные атрибуты не раскрываются в отдельные отношения, а хранятся в полях типа `jsonb`. В модели используются следующие JSONB-поля:

- `camera.position` — положение и ориентация камеры;
- `camera.settings` — настройки видеопотока и аналитики;
- `event.payload` — параметры события, зависящие от его типа;
- `camera_telemetry.metrics` — технические показатели камеры.

Обнаруженные объекты не выделяются в отдельную таблицу этой модели. Они хранятся внутри `event.payload` как массив объектов события. Стабильная часть модели остается реляционной, а данные с переменной структурой хранятся как документы внутри PostgreSQL.

3.1.3. Особенности модели

PostgreSQL JSONB-модель позволяет оценить компромисс между реляционным хранением и документным представлением вложенных данных. При записи события не требуется раскладывать все параметры события и обнаруженные объекты по множеству таблиц, однако запросы к внутренним полям `jsonb` требуют специальных индексов и отличаются от обычных реляционных соединений.

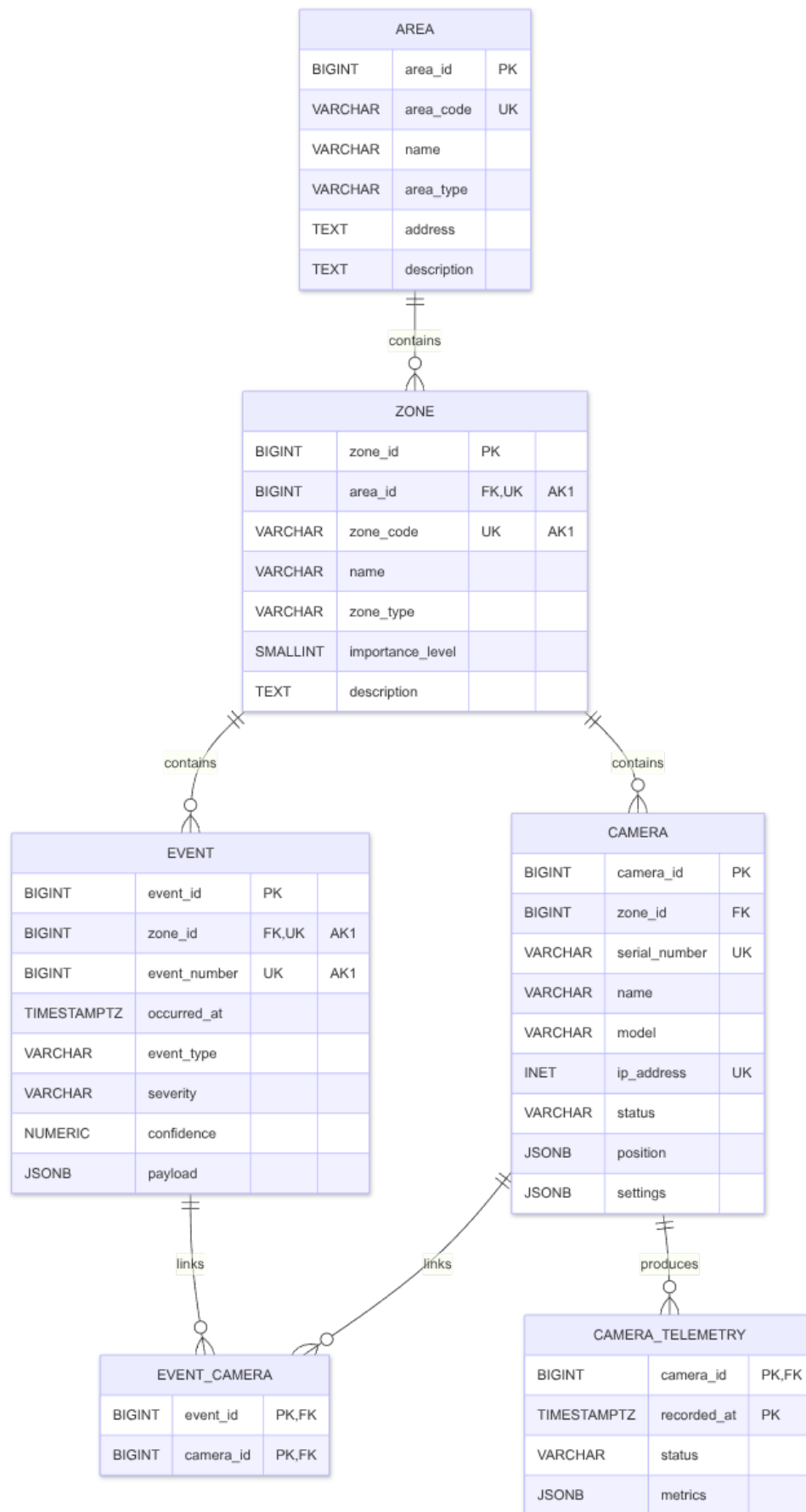


Рисунок 3.1 – Реляционная JSONB-модель

3.2. Нормализованная реляционная модель PostgreSQL

Полностью нормализованная реляционная модель строится на основе PostgreSQL JSONB-модели. Основные сущности, связи, суррогатные первичные ключи, альтернативные ключи и внешние ключи сохраняются. Заметно изменяется способ хранения составных и вариативных данных: поля, которые в JSONB-модели находились внутри `jsonb`, раскрываются в столбцы и отдельные отношения.

3.2.1. Нормализация справочных значений

Сначала из отношений выделяются домены допустимых значений. Тип территории хранится в `area_type`, тип зоны — в `zone_type`, состояние камеры — в `camera_status`, тип события — в `event_type`, уровень значимости события — в `event_severity`. По тому же правилу выделяются `object_type`, `direction`, `video_codec` и `signal_lost_reason`.

Такие таблицы выделяются при устранении транзитивных зависимостей, что соответствует приведению схемы к третьей нормальной форме. Сам по себе код статуса или типа не нарушает нормальную форму. Проблема возникает, если в основной таблице вместе с кодом хранить его название, описание или ранг: эти поля зависят уже не от ключа основной таблицы, а от справочного кода. Например, идентификатор камеры определяет статус, а статус определяет его название. Поэтому справочные атрибуты переносятся в отдельные отношения, а в основных таблицах остаются внешние ключи. Такие отношения также задают допустимые значения полей.

3.2.2. Декомпозиция составных атрибутов

Дальше раскладываются составные и многозначные атрибуты:

- `camera.position` не образует отдельного отношения, так как описывает одну камеру и содержит фиксированный набор значений. Поэтому координаты установки и углы ориентации переносятся в столбцы таблицы `camera`;
- `camera.settings` содержит несколько логически разных групп. Параметры видеопотока переходят в `camera_stream_setting`, настройки аналитики — в `camera_analytics_setting`. Области детекции становятся отношением `camera_detection_zone`, а точки контура этих областей — отношением `camera_detection_zone_point`. Линии пересечения выносятся в `camera_crossing_line`;
- `event.payload` зависит от типа события, поэтому общая часть события остается в `event`, а параметры, зависящие от типа события, переносятся в отношения `motion_event_detail`, `object_detection_event_detail`, `line_crossing_event_detail` и `signal_lost_event_detail`;
- массив `event.payload.objects` является многозначным атрибутом события. Его элементы хранятся в `detected_object`. Атрибуты, зависящие от типа объекта, вынесены в `person_object_attribute` и `vehicle_object_attribute`;
- `camera_telemetry.metrics` содержит фиксированный набор технических показателей, поэтому раскрывается в столбцы таблицы `camera_telemetry`: температуру, загрузку процессора, использование памяти, битрейт, потери пакетов, задержку и время непрерывной работы.

3.2.3. Нормальная форма

Декомпозиция выполняется ради устранения зависимостей, которые затрудняют обновление данных и приводят к их дублированию. На пер-

вом шаге схема приводится к первой нормальной форме: атрибуты становятся атомарными относительно выбранной реляционной модели, поэтому `event.payload.objects` и точки областей детекции выносятся в отдельные отношения.

Затем проверяется зависимость неключевых атрибутов от ключей. Для отношений с составными ключами неключевые атрибуты должны зависеть от всего ключа, а не от его части. Например, в таблице связи `event_camera` нет собственных описательных атрибутов камеры или события: они остаются в `camera` и `event`. Это соответствует требованию второй нормальной формы.

Третья нормальная форма требует, чтобы неключевые атрибуты не зависели от других неключевых атрибутов. Поэтому названия, описания и ранги справочных значений вынесены из основных таблиц в `area_type`, `camera_status`, `event_type`, `event_severity` и другие справочники. В основной таблице остается только ссылка на справочное значение.

С учетом заданных ключей и выделенных таблиц отношения могут рассматриваться как удовлетворяющие более строгому условию BCNF: в каждой нетривиальной функциональной зависимости детерминант является суперключом своего отношения. Иначе говоря, атрибуты в каждой таблице описывают только объект, заданный ее ключом, а данные о других объектах вынесены в собственные отношения.

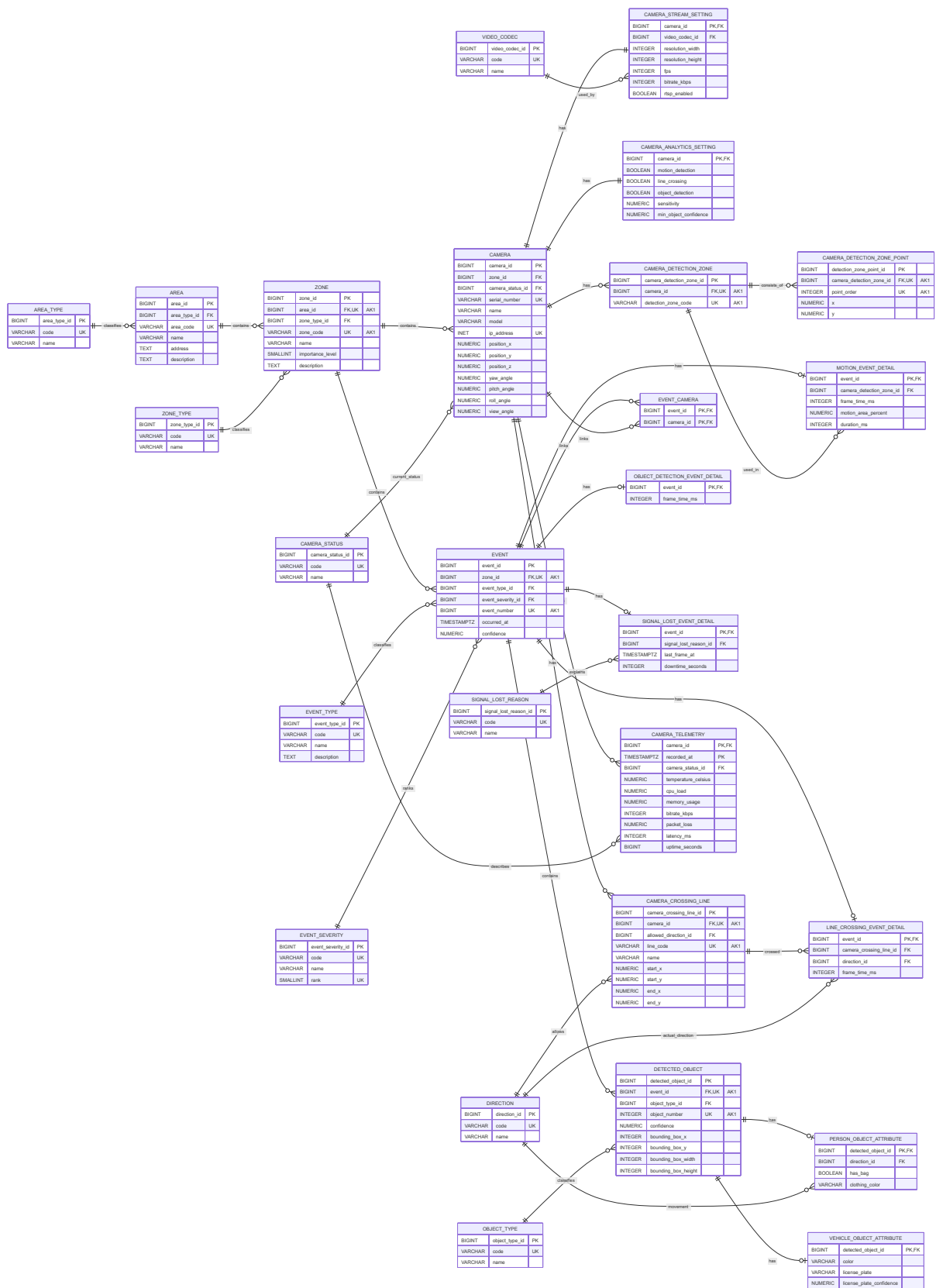


Рисунок 3.2 – Нормализованная реляционная модель

3.3. Вложенная документная модель MongoDB

Вложенная документная модель MongoDB строится вокруг данных, которые обычно читаются вместе. Для нее используются четыре коллекции: `areas`, `cameras`, `events` и `camera_telemetry`.

В коллекции `areas` документ территории содержит массив `zones`. Зоны вложены в территорию, потому что они не имеют самостоятельного потока записей и всегда принадлежат одной территории.

Камеры вынесены в коллекцию `cameras`. Камера не встраивается в зону, так как она используется не только при описании территории, но и в событиях, и в телеметрии. Связь камеры с местом установки хранится внутри документа камеры: в него встроены краткие сведения о территории и зоне. В тот же документ помещены `position` и `settings`, поскольку положение камеры и настройки обычно читаются вместе с самой камерой.

События хранятся в коллекции `events`. В документ события встроены территория, зона, список камер, параметры события `payload` и массив обнаруженных объектов `payload.objects`. Поэтому событие можно читать сразу вместе с контекстом, не восстанавливая его через отдельные коллекции.

Телеметрия хранится в отдельной коллекции `camera_telemetry`, потому что это быстро растущий временной ряд. Каждая запись телеметрии содержит краткие сведения о камере, территории и зоне, а технические показатели помещены во вложенный документ `metrics`. Такое дублирование увеличивает объем данных, но уменьшает число обращений при чтении.

3.4. Нормализованная документная модель MongoDB

Нормализованная документная модель MongoDB хранит те же данные, но разделяет сущности по коллекциям и связывает их ссылками `$ref/$id`. Для этой модели используются коллекции:

- `areas` — территории;
- `zones` — зоны со ссылкой на территорию;
- `cameras` — камеры со ссылкой на зону;
- `events` — события со ссылкой на зону;
- `event_cameras` — связи событий и камер;
- `camera_telemetry` — телеметрия со ссылкой на камеру.

Связь многие-ко-многим между событиями и камерами вынесена в `event_cameras`. Это уменьшает дублирование сведений о камерах по сравнению с вложенной моделью.

Составные поля при этом остаются документами: `camera.position`, `camera.settings`, `event.payload` и `camera_telemetry.metrics`. Поэтому модель сопоставима с PostgreSQL JSONB: связи нормализованы, а вариативные параметры события и телеметрии остаются вложенными структурами.

Для аналитических запросов такая схема требует дополнительных переходов по ссылкам. Например, чтобы сгруппировать события по территории и зоне, запросу нужно восстановить контекст через `zones` и `areas`. Это и создает основную стоимость нормализованной MongoDB-модели в эксперименте.

4. Реализация баз данных и веб-приложения

4.1. Проектирование программной системы

4.1.1. Общая архитектура

Для проведения эксперимента была реализована программная система из нескольких контейнеров. Пользователь работает с Vue-приложением, выбирает модель хранения, профиль данных и сценарий нагрузки. Клиентское приложение отправляет запросы в Express API, а сервер перенаправляет их в нужный экземпляр сервиса генерации нагрузки (loadgen).

В системе постоянно запущены два loadgen-сервиса. Первый подключается к PostgreSQL и выполняет прогоны для моделей pg-jsonb и pg-normalized. Второй подключается к MongoDB и выполняет прогоны для mongo-nested и mongo-normalized. Такой вариант позволяет использовать общий код генератора нагрузки, но разделить подключение к разным СУБД.

Метрики собираются через Prometheus. Loadgen публикует собственные метрики операций, PostgreSQL exporter и MongoDB exporter показывают состояние баз данных, а Grafana отображает итоговые графики. Архитектура приложения показана на рисунке.

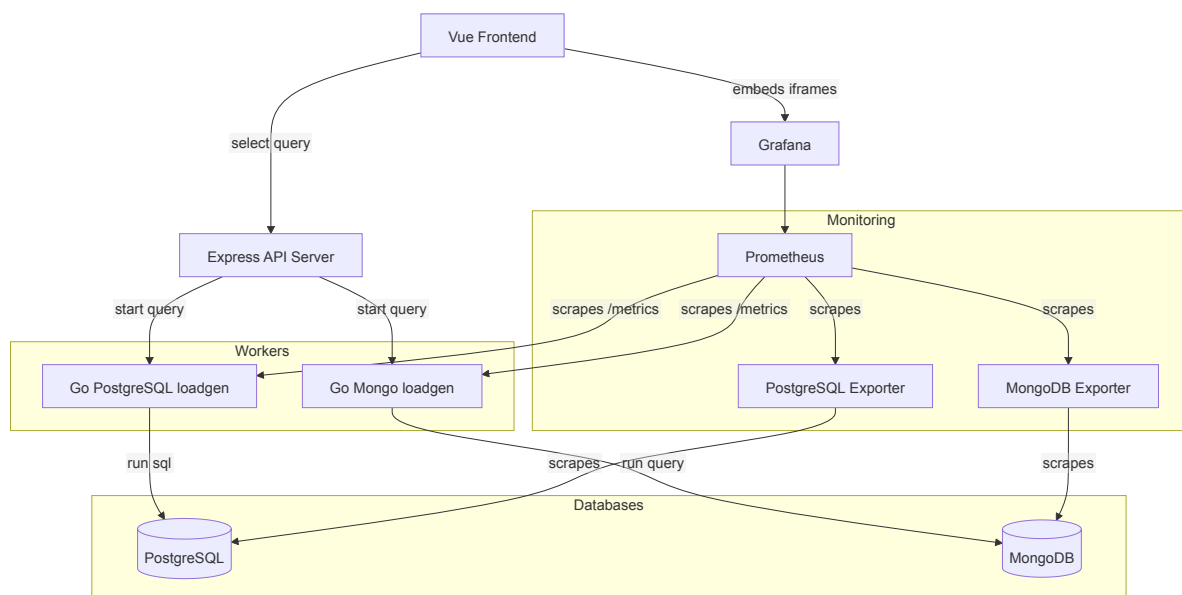


Рисунок 4.1 – Архитектура веб-приложения

4.2. Создание баз данных

4.2.1. Инициализация схем

Схемы баз данных создаются заранее с помощью init-скриптов, а во время эксперимента сервис создания нагрузки (loadgen) работает только с данными выбранной модели.

В PostgreSQL две модели размещены в одной базе, но в разных схемах: jsonb и normalized. В MongoDB модели разделены по базам: coursework_nested и coursework_normalized. Такое разделение не смешивает данные разных моделей при последовательных запусках.

4.2.2. Индексы

Индексы подбирались под операции нагрузки. Для чтения за временной интервал используются индексы по времени события и телеметрии: occurred_at и recorded_at. Для выборки по месту наблюдения добавлены составные индексы по территории, зоне и времени: area_code, zone_code, occurred_at или recorded_at. Для поиска важных событий используются ин-

дексы по типу события, уровню значимости и времени.

В нормализованных моделях дополнительно индексируются поля связей: `zone_id`, `camera_id`, `event_id` и соответствующие `$ref`-ссылки в MongoDB. В PostgreSQL JSONB добавлены GIN-индексы по `payload`, `settings` и `metrics`. В документных моделях MongoDB индексируются вложенные поля, которые участвуют в запросах: `payload.objects.object_type`, `area.area_code`, `zone.zone_code`, `camera.serial_number`.

4.2.3. Подготовка данных

Перед запуском нагрузки создается базовый мир видеонаблюдения: территории, зоны, камеры, положения камер, настройки видеопотока, настройки аналитики, области детекции и линии пересечения. Генерация детерминированная: одинаковые `seed` и профиль размера дают одинаковые предметные данные для всех моделей. События и телеметрия создаются уже во время нагрузочного цикла, поскольку именно они образуют основной поток записей.

Для экспериментов предусмотрены профили `small`, `medium` и `large`. В запуске из веб-интерфейса по умолчанию используется `small`: 5 территорий, 5 зон на территорию и 5 камер на зону.

4.3. Реализация симулятора

4.3.1. Генератор нагрузки

Loadgen — сервис создания нагрузки, реализованный на Go. Он подключается к PostgreSQL или MongoDB, заполняет исходные данные и выполняет операции нагрузки по выбранному сценарию без использования ORM.

Генератор предоставляет HTTP API: `/health`, `/metrics`, `/clear`, `/seed` и `/run`. Express API вызывает эти методы от имени веб-приложения. Для каж-

дого запуска формируется `run_id`, по которому Grafana отделяет метрики одного прогона от другого.

4.3.2. Метрики `loadgen`

Через `/metrics loadgen` отдает метрики Prometheus. Для каждой метрики используются метки `run_id`, `model`, `scenario`, `operation` и `stage_clients`. Это позволяет сравнивать модели, сценарии и ступени параллелизма на одной панели Grafana.

В эксперименте используются следующие метрики:

- `loadgen_operation_duration_seconds` — гистограмма времени выполнения операции, по ней считаются p50, p95 и p99;
- `loadgen_operation_total` — число выполненных операций с разделением на успешные и ошибочные;
- `loadgen_active_workers` — текущее число активных клиентов на ступени нагрузки;
- `loadgen_batch_size` — размер пакета для операций записи;
- `loadgen_operation_rows_total` — количество обработанных строк или документов;
- `loadgen_operation_bytes_total` — примерный объем обработанных данных.

4.3.3. Операции нагрузки

В эксперименте используются пять операций:

- `WRITE_COMPLEX_EVENT_BATCH` — пакетная запись аналитических событий с различным `payload` и массивом обнаруженных объектов;

- `WRITE_TELEMETRY_STREAM` — потоковая запись технической телеметрии камер;
- `AGG_OBJECT_ACTIVITY_BY_AREA` — подсчет обнаруженных объектов по территории, зонам, типам объектов и уровню значимости;
- `AGG_TELEMETRY_HEALTH_WINDOW` — расчет технического состояния камер за заданный интервал времени;
- `READ_INCIDENT_TIMELINE` — чтение списка важных событий в зоне вместе с камерами, объектами и телеметрией.

Для сравнения используются три профиля нагрузки:

- `write-heavy`: 60% записи событий, 35% записи телеметрии, 5% операций чтения и аналитики;
- `analytics-heavy`: 30% записи и 70% чтения с аналитическими агрегациями;
- `balanced`: 70% записи и 30% чтения, в том числе по 10% на каждую аналитическую операцию.

Профиль наполняется через веса операций. На каждой итерации клиент `loadgen` выбирает одну из пяти операций по заданным долям: например, в `write-heavy` первые 60% выборов приходятся на запись событий, следующие 35% — на запись телеметрии, а оставшиеся 5% — на чтение. Данные для операций берутся из подготовленного мира наблюдения: территорий, зон и камер. Новые события и записи телеметрии создаются уже во время нагрузочного цикла.

Конкретные параметры операции выбираются из того же набора данных. При записи событий `loadgen` выбирает зону, одну из камер этой зоны, тип события, значимость и содержимое `payload`. При записи телеметрии

выбирается камера и формируется набор технических показателей. Для агрегаций выбирается территория, а для чтения важных событий — зона; во всех запросах чтения используется заданный интервал времени вокруг момента выполнения операции.

В каждом профиле нагрузка выполняется ступенями по 60 секунд. Число параллельных клиентов последовательно увеличивается: 1, 5, 10 и 25. Размер пакета для событий равен 25, для телеметрии — 50.

4.3.4. Характер запросов

Операции нагрузки моделируют типовые операции в системе видеонаблюдения: запись новых наблюдений, подсчет обнаруженных объектов, проверку технического состояния камер и поиск важных событий в выбранной зоне.

К примеру, при записи события в базу попадает не только строка о самом событии, но и связанные с ним камеры, уровень значимости, тип события и параметры `payload`. В PostgreSQL JSONB вариативная часть сохраняется одним полем `payload`. В нормализованной PostgreSQL то же событие раскладывается на основную таблицу, таблицу связи с камерами, таблицы деталей события и таблицу обнаруженных объектов. Поэтому эта операция показывает, насколько дорого записывать сложное событие.

Характер аналитической нагрузки можно показать на операции `AGG_OBJECT_ACTIVITY_BY_AREA`. Она отвечает на вопрос: какие объекты фиксировались на выбранной территории за заданный интервал времени. Результат группируется по зоне, типу объекта и уровню значимости события. Ниже приведен вариант этого запроса для PostgreSQL JSONB:

```
SELECT z.zone_code, obj->>'object_type', e.severity,  
       count(*), avg((obj->>'confidence')::numeric)  
FROM jsonb.event e  
JOIN jsonb.zone z ON z.zone_id = e.zone_id
```

```

JOIN jsonb.area a ON a.area_id = z.area_id
CROSS JOIN LATERAL jsonb_array_elements(e.payload->'objects') obj
WHERE a.area_code = $1 AND e.occurred_at BETWEEN $2 AND $3
GROUP BY z.zone_code, obj->>'object_type', e.severity;

```

Для вложенной MongoDB та же операция выполняется по коллекции events. Так как территория, зона и обнаруженные объекты уже лежат внутри документа события, конвейер состоит из фильтрации, раскрытия массива payload.objects и группировки:

```

[
  { $match: { "area.area_code": areaCode,
             occurred_at: { $gte: from, $lte: to } } },
  { $unwind: "$payload.objects" },
  { $group: {
    _id: { zone: "$zone.zone_code",
          object_type: "$payload.objects.object_type",
          severity: "$severity" },
    count: { $sum: 1 },
    avg_confidence: { $avg: "$payload.objects.confidence" }
  } }
]

```

В этом примере нагрузку создает не только фильтрация по территории и времени, но и разбор массива payload.objects. В нормализованных моделях такой же запрос требует дополнительно восстановить связи между событиями, зонами и территориями; в нормализованной MongoDB для этого используются этапы \$lookup.

Остальные операции чтения имеют тот же характер: они берут данные за заданный интервал времени и связывают их с территорией, зоной или камерой. Запрос по телеметрии считает задержку, потери пакетов, температуру и долю записей со статусом signal_lost. Запрос по важным событиям выбирает последние события высокой значимости в зоне и читает связанный с ними контекст камер.

4.4. Реализация веб-приложения

4.4.1. Пользовательский интерфейс

Веб-приложение используется для запуска эксперимента. Пользователь выбирает модель хранения, профиль нагрузки, seed, длительность ступени, число параллельных клиентов и размеры пакетов. После этого приложение выполняет очистку, подготовку данных и нагрузочный прогон.

Все команды проходят через Express API. Это позволяет запускать только один прогон за раз и не смешивать результаты разных моделей.

4.4.2. Отображение метрик

Для просмотра результатов в интерфейс встроена панель Grafana [4]. В отчете используются следующие показатели: общее количество операций, доля ошибок, пропускная способность, объем обработанных данных, p95 и p99 задержек, пропускная способность по моделям и ступеням нагрузки, а также задержки операций записи и чтения. Эти метрики СУБД используются для построения вывода о производительности конкретной модели хранения данных. Метрики сравниваются между собой на разных моделях на разных уровнях нагрузки и сценариях.

5. Экспериментальное сравнение моделей

5.1. Сравнение времени записи

Эксперимент запускался последовательно для четырех моделей хранения: pg-jsonb, pg-normalized, mongo-nested и mongo-normalized. Для всех запусков использовались одинаковые параметры: профиль данных small, seed = 42, ступени параллелизма 1, 5, 10 и 25 клиентов, длительность

каждой ступени 60 секунд. Порядок запуска был фиксированным: сначала PostgreSQL JSONB, затем нормализованная PostgreSQL, вложенная MongoDB и нормализованная MongoDB.

Запись проверялась двумя операциями: пакетной записью сложных событий `WRITE_COMPLEX_EVENT_BATCH` и потоковой записью телеметрии `WRITE_TELEMETRY_STREAM`. В сценарии `write-heavy` на эти операции приходится 95% нагрузки, поэтому он лучше всего показывает цену вставки данных в разные модели.

На рисунке 5.1 видно, что за прогон `write-heavy` выполнено около 113 тыс. операций, обработано около 1.7 GiB данных, ошибки не зафиксированы. Самую высокую пропускную способность показала вложенная MongoDB-модель: пик на графике пропускной способности по моделям превышает 300 операций в секунду. PostgreSQL JSONB и нормализованная PostgreSQL держатся ниже, но остаются устойчивыми. Нормализованная MongoDB заметно отстает по пропускной способности и дает более длинные хвостовые задержки.

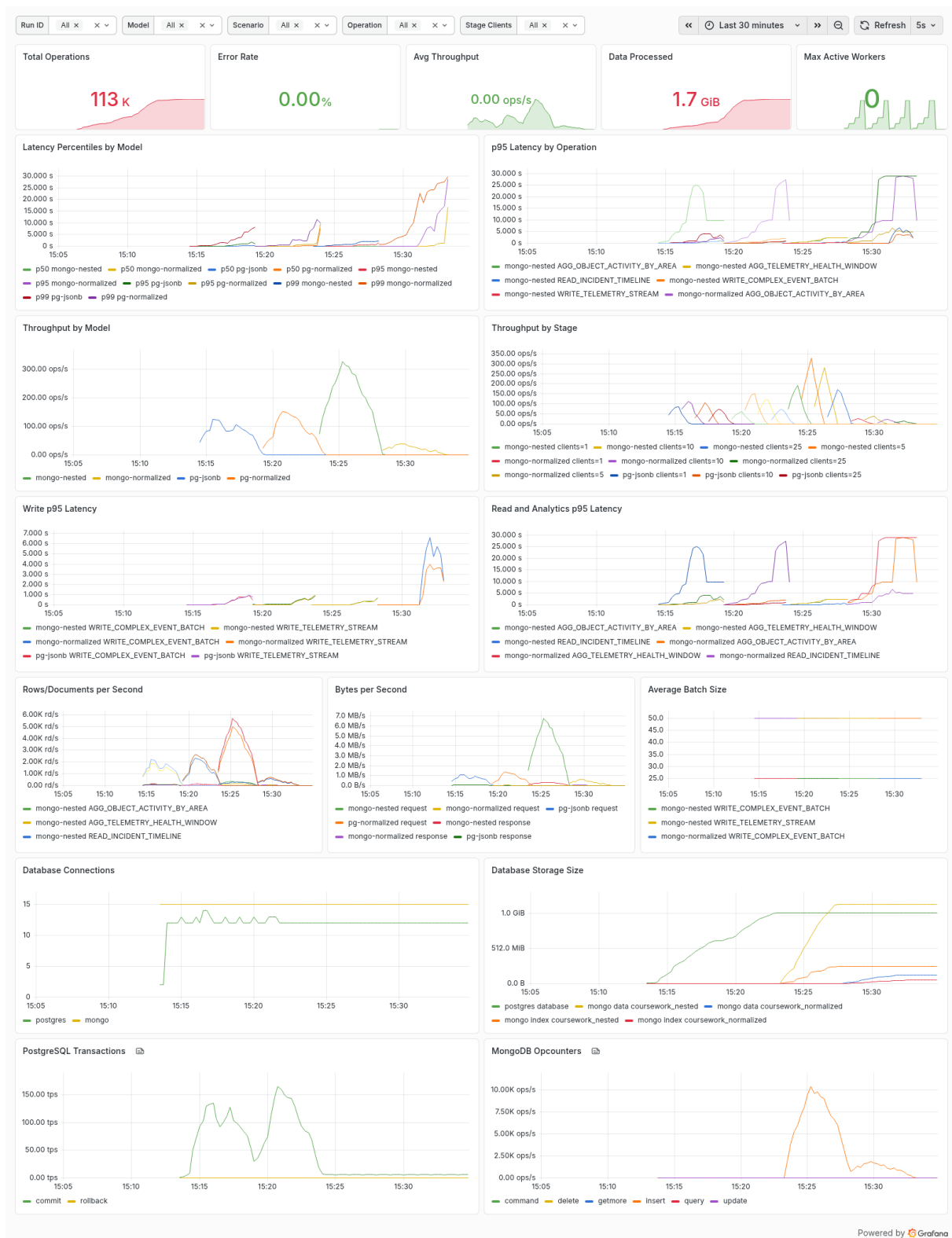


Рисунок 5.1 – Панель Grafana для сценария write-heavy

5.2. Сравнение времени чтения

Чтение проверялось тремя операциями: агрегацией активности объектов, агрегацией технического состояния камер и чтением таймлайна инцидента. Сценарий *analytics-heavy* отдает этим операциям 70% нагрузки, поэтому лучше показывает стоимость аналитических запросов.

За прогон *analytics-heavy* выполнено около 71 тыс. операций и обработано около 812.6 MiB данных. На рисунке 5.2 видно, что вложенная MongoDB-модель и нормализованная PostgreSQL дают наибольшую пропускную способность в начале и середине прогона: примерно 140 и 90–100 операций в секунду соответственно. PostgreSQL JSONB показывает хорошую скорость на отдельных ступенях, но задержки аналитических операций растут при увеличении числа клиентов. Нормализованная MongoDB снова имеет самую низкую пропускную способность, потому что аналитические запросы вынуждены восстанавливать контекст через ссылки между коллекциями.

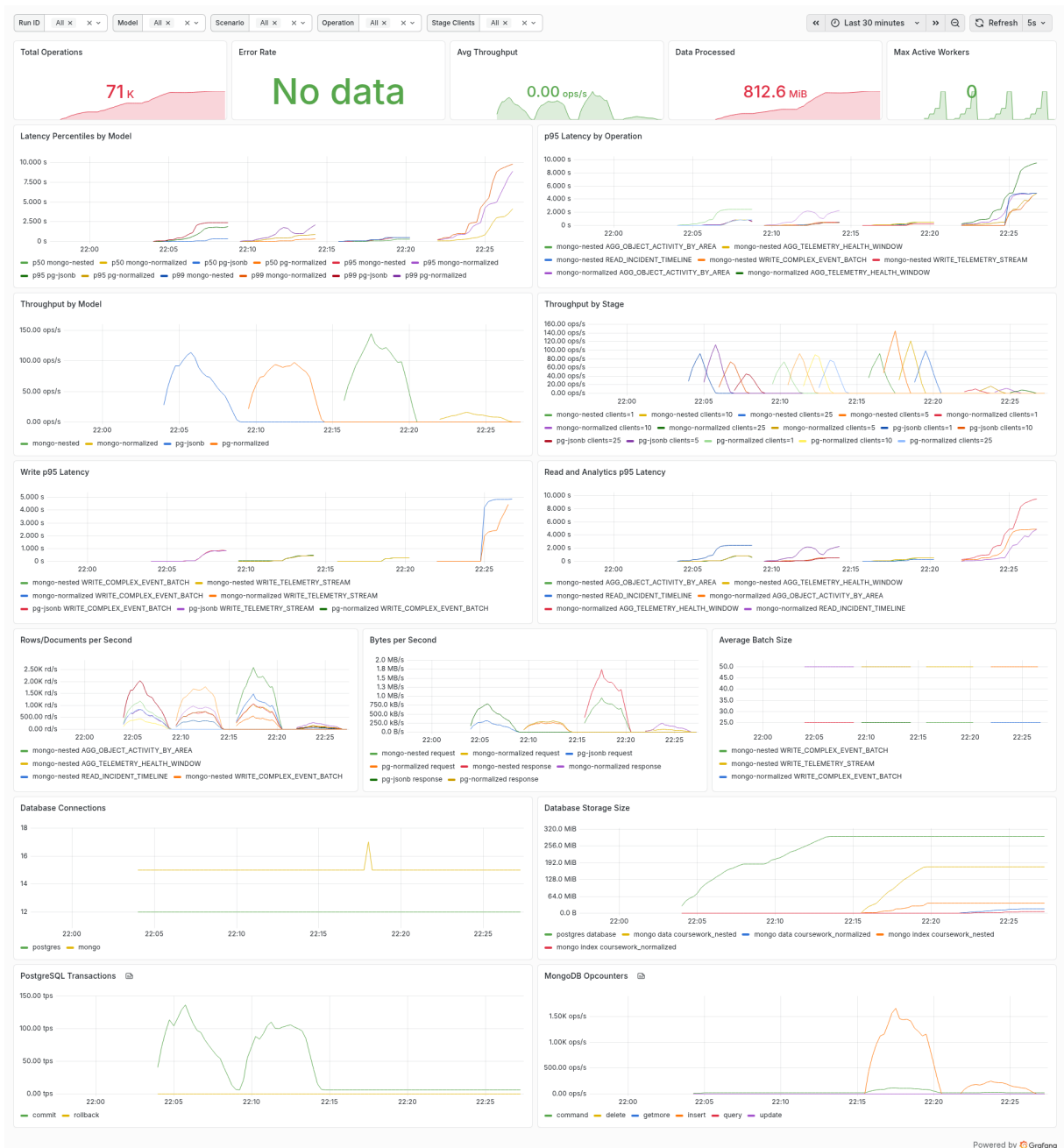


Рисунок 5.2 – Панель Grafana для сценария analytics-heavy

5.3. Сравнение моделей при разной нагрузке

Сценарий `balanced` используется как смешанный режим: 70% операций связано с записью, 30% – с чтением и аналитикой. Рост нагрузки задавался степенями параллелизма. На каждой ступени `loadgen` запускал одинаковую смесь операций, а Grafana показывала пропускную способность и проценты задержек по модели, операции и числу клиентов.

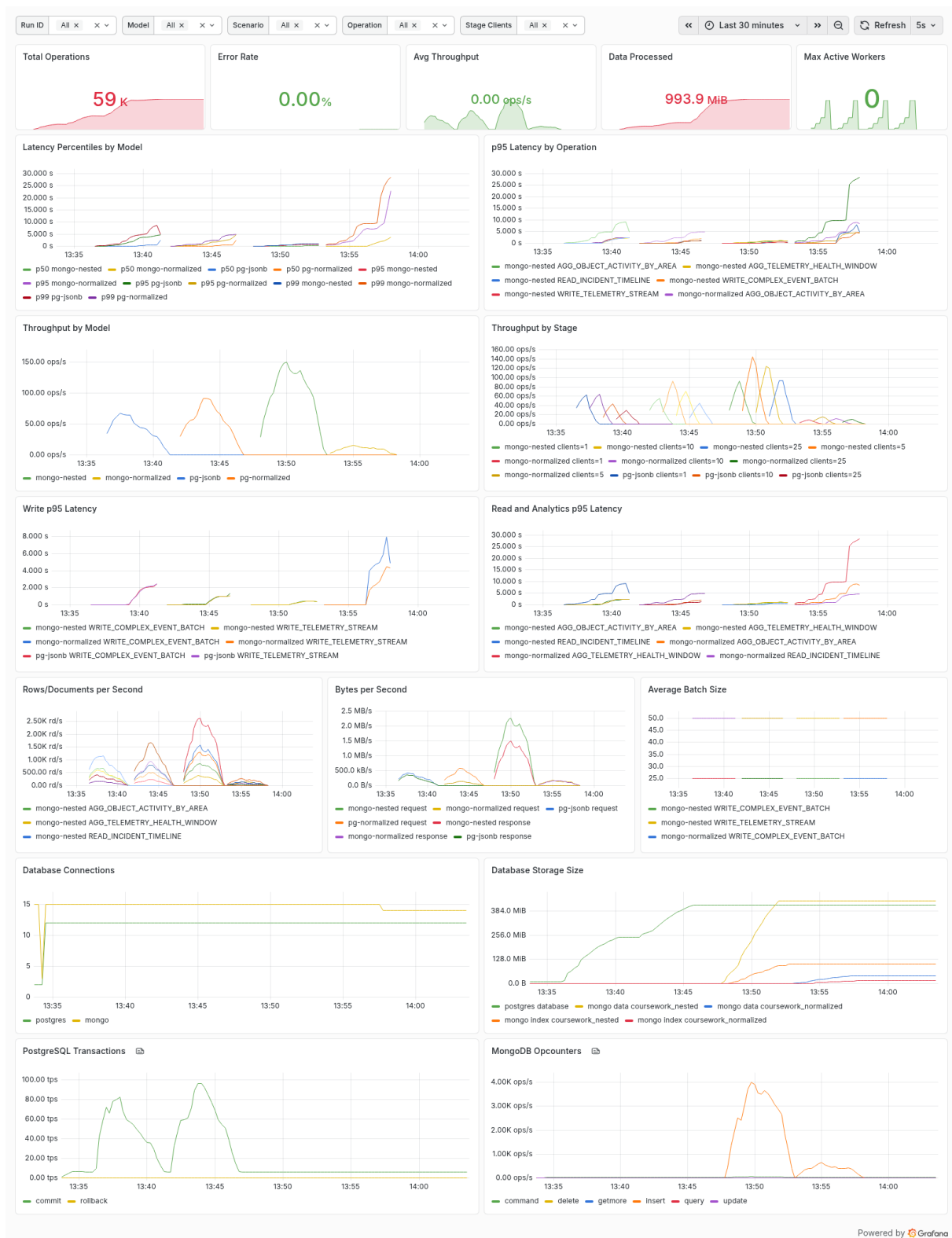


Рисунок 5.3 – Панель Grafana после последовательного запуска моделей в сценарии balanced

За один последовательный запуск было выполнено около 59 тыс. операций, доля ошибок составила 0%, объем обработанных данных – около 994 MiB. Пиковая пропускная способность различалась по моделям. На

графике пропускной способности по моделям вложенная MongoDB достигает примерно 150 операций в секунду, нормализованная PostgreSQL – около 90 операций в секунду, PostgreSQL JSONB – около 60 операций в секунду. Нормализованная MongoDB в этом запуске показала меньшую пропускную способность и более длинные хвостовые задержки: p95 отдельных аналитических операций поднимается примерно до 25–30 секунд.

5.4. Анализ полученных результатов

После всех экспериментов лучший результат показала вложенная MongoDB. В сценарии *write-heavy* она дала наибольшую пропускную способность при записи событий и телеметрии. Это связано с тем, что сложное событие сохраняется как один документ вместе со всем своим контекстом, а не раскладывается на несколько таблиц или коллекций.

В сценариях *analytics-heavy* и *balanced* вложенная MongoDB также осталась лучшей по общему поведению. Для запросов системы видеонаблюдения часто требуется получить событие вместе с территорией, зоной, камерами и обнаруженными объектами. Во вложенной модели эти данные уже лежат рядом, поэтому запросу нужно меньше соединений и переходов по ссылкам.

Нормализованная PostgreSQL показала себя лучше нормализованной MongoDB: реляционные связи и типизированные поля дают более устойчивое выполнение аналитических запросов. PostgreSQL JSONB заняла промежуточное положение: вариативный *payload* удобно записывать, но агрегации по массивам внутри JSONB дороже работы с обычными столбцами. Нормализованная MongoDB оказалась самым слабым вариантом для аналитики: при чтении ей приходится восстанавливать контекст через *\$lookup*.

Индексы были созданы во всех моделях по полям, которые участ-

вуют в фильтрации и связях: времени, территории, зоне, типу события, уровню значимости и идентификаторам связанных сущностей. Поэтому результат нельзя объяснить тем, что одна модель была проиндексирована, а другая нет. Разница возникла из-за самой структуры хранения: вложенная MongoDB лучше соответствует тем запросам, которые выполнялись в эксперименте.

Заключение

В ходе работы были спроектированы четыре модели хранения данных системы видеонаблюдения: PostgreSQL JSONB, нормализованная PostgreSQL, вложенная MongoDB и нормализованная MongoDB. Все модели описывают одни и те же данные: территории, зоны, камеры, аналитические события и телеметрию.

Для сравнения этих моделей был реализован стенд нагрузочного тестирования. Для запуска прогонов и просмотра метрик использовалось веб-приложение. Сравнение моделей выполнялось по времени записи, времени чтения и пропускной способности.

Лучший результат по сумме метрик показала вложенная MongoDB: она дала наибольшую пропускную способность и особенно хорошо показала себя при чтении. PostgreSQL JSONB не обогнала вложенную MongoDB, но была немного быстрее нормализованной PostgreSQL как на запись, так и на чтение. Нормализованная MongoDB уменьшила дублирование данных, однако значительно проиграла всем остальным моделям по производительности.

Таким образом, в проведенных экспериментах выиграла вложенная документная модель MongoDB. Главная причина – денормализация данных, которые обычно читаются вместе в сценариях видеонаблюдения. При аналитических запросах системе не нужно соединять несколько сущностей, а при записи сложного события не нужно раскладывать его на несколько таблиц. За счет этого уменьшаются накладные расходы, растет пропускная способность и снижаются задержки.

Источники

- [1] U.S. Department of Homeland Security. Closed circuit television: Technical handbook, July 2013. URL https://www.dhs.gov/sites/default/files/publications/CCTV-Tech-HBK_0713-508.pdf.
- [2] Вишняков И. Э. Лекции по базам данных, 2025.
- [3] Postgresql documentation, 2026. URL <https://www.postgresql.org/docs/current/index.html>.
- [4] Grafana database observability, 2026. URL <https://grafana.com/docs/grafana-cloud/monitor-applications/database-observability/monitor/>.
- [5] MongoDB documentation, 2026. URL <https://www.mongodb.com/docs/>.